MyCodeSchool

# Teorie — șiruri de caractere

Python · proiectul `py-siruri` · Luca

## 1. Ce e un șir

Text pus între ghilimele: `"Python"` sau `'Python'` — e același lucru. Fiecare literă are o poziție, numărată de la **0**, și una de la coadă, cu minus:

| poziție | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| literă | P | y | t | h | o | n |
| de la coadă | −6 | −5 | −4 | −3 | −2 | −1 |

`len("Python")` e `6`, deci ultima poziție e `5`, nu `6`.

## 2. O literă și o felie

Pentru toate exemplele, `text = "Python"`.

| scrii | primești | regula |
|---|---|---|
| `text[0]` | `"P"` | o literă, după poziție |
| `text[-1]` | `"n"` | ultima, oricât de lung e textul |
| `text[1:4]` | `"yth"` | de la 1 până **înainte** de 4 |
| `text[:2]` | `"Py"` | lipsește începutul → de la 0 |
| `text[2:]` | `"thon"` | lipsește sfârșitul → până la capăt |
| `text[::-1]` | `"nohtyP"` | tot, de la coadă |

**Diferența care contează:** o poziție care nu există dă eroare — `text[6]` → `IndexError`, la fel `""[0]`. O felie nu dă niciodată eroare: `text[:100]` e pur și simplu `"Python"`.

## 3. Parcurgerea

`for` îți dă pe rând fiecare literă — fără poziții, fără contor:

```
for litera in "abc":
    print(litera)
```

Iar pe cuvinte, cu `.split()`:

```
for cuvant in "ana are mere".split():
    print(cuvant)
```

Cele două tipare de la cifre funcționează la fel și aici:

| tipar | pornești de la | în buclă |
|---|---|---|
| contor | `contor = 0` | `contor += 1` |
| acumulator de text | `rezultat = ""` | `rezultat += litera` |

## 4. Metodele de care ai nevoie

| scrii | primești |
|---|---|
| `"Ana".lower() / "Ana".upper()` | `"ana" / "ANA"` |
| `"  ana are  ".strip()` | `"ana are"` |
| `"ana are mere".split()` | `["ana", "are", "mere"]` |
| `"ana,are,mere".split(",")` | `["ana", "are", "mere"]` |
| `"a-b-c".replace("-", "")` | `"abc"` |
| `"banana".count("a")` | `3` |
| `"Py" in "Python"` | `True` |

**Un șir nu se modifică niciodată.** Metodele **întorc** un șir nou; dacă nu-l păstrezi, s-a pierdut:

```
nume = "luca"
nume.upper()          # nume a rămas "luca"
nume = nume.upper()   # acum nume e "LUCA"
```

## 5. Text și numere

| scrii | primești | de ce |
|---|---|---|
| `"5" + "3"` | `"53"` | `+` pe text lipește |
| `"5" + 3` | `TypeError` | text și număr nu se adună |
| `int("25") + 1` | `26` | text → număr |
| `str(4738)` | `"4738"` | număr → text |
| `"ab" * 3` | `"ababab"` | repetă |
| `"Ana" == "ana"` | `False` | literele mari contează |

`input()` întoarce **mereu text**, chiar dacă tastezi un număr — de aceea scrii `int(input())`. Iar ca să pui variabile în text, f-string: `f"{text} are {len(text)} litere"` → `Python are 6 litere`.

**Greșelile pe care le vei face — verifică-le întâi:**

1. `text[len(text)]` → `IndexError`. Ultima literă e `text[-1]`.
2. `text.lower()` scris singur pe rând → nu schimbă nimic. Scrie `text = text.lower()`.
3. `text[0] = "J"` → `TypeError`. Un șir nu se modifică: construiești altul.
4. Ai uitat că `input()` dă text → `"5" + "3"` e `"53"`, nu `8`.